DevDocs — Search & Retrieval

1. Purpose

Search & Retrieval is responsible for finding the most relevant pieces of a project's knowledge before the LLM generates an answer.

The user asks a question, and DevDocs searches the current project's indexed chunks to find the information most relevant to that question.

```
User Question
      ↓
Search & Retrieval
      ↓
Relevant Chunks
      ↓
RAG / Answer Generation
      ↓
Answer + Sources
```

Search & Retrieval is different from the **Document Explorer file search**.

Document Explorer search

Answers:

> "Where is `auth.py`?"

It searches filenames and paths.

Search & Retrieval

Answers:

> "How does authentication work?"

It searches the actual indexed project knowledge using semantic and keyword-based retrieval.

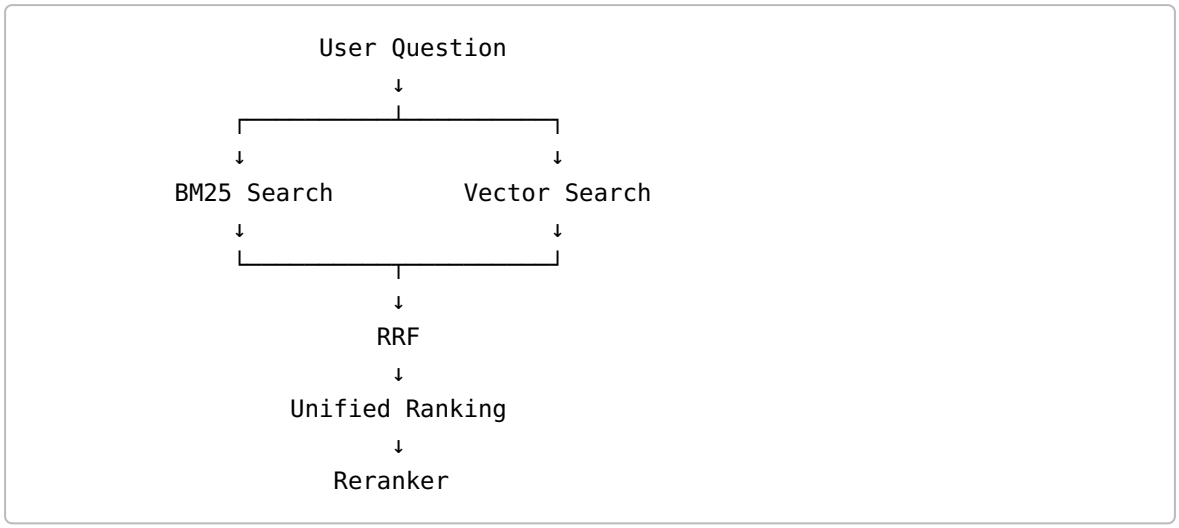
---

2. Hybrid Retrieval

DevDocs will use **two retrieval strategies together**:

1. BM25 / keyword search
2. Vector / semantic search

This is called **Hybrid Search**.



The reason for using both is that developer questions can depend on either:

- exact technical terminology
- semantic meaning

---

3. BM25 / Keyword Search

BM25 is a smart keyword-based search strategy.

It looks for important terms from the user's question in the indexed chunks and ranks chunks according to how strongly they match those terms.

For example, the user asks:

> "Where is `validate_token()` called?"

BM25 is useful because the exact identifier:

```
validate_token
```

is important.

It can find chunks such as:

```
validate_token(token)
```

BM25 is particularly useful for

- Function names

- Class names
- Variable names
- API endpoints
- Error messages
- Configuration keys
- Exact technical terminology
- File-related terms

Simple mental model

> **BM25 = find chunks containing important words/terms from the question.**

---

4. Vector Search

Vector search focuses on **semantic meaning** rather than exact words.

The user's question is converted into an embedding vector.

The indexed chunks already have their own embedding vectors.

The system compares the question vector with chunk vectors and retrieves semantically similar chunks.

```
User Question
      ↓
Embedding
      ↓
Question Vector
      ↓
Compare with Chunk Vectors
      ↓
Most Similar Chunks
```

For example, the user asks:

> "Where does the application verify the user's identity?"

The code might contain:

```
def authenticate_user(username, password):
    ...
```

Even though the exact words in the question and code differ, the concepts are related.

Vector search can therefore find semantically relevant content that keyword search might miss.

Vector search is particularly useful for

- Conceptual questions
- Questions using different wording from the source
- Understanding relationships between concepts
- Natural-language questions
- Questions where the user does not know the project's terminology

Simple mental model

> **Vector search = find chunks that mean something similar to the question.**

---

5. Why DevDocs Uses Both

Developer questions can be different types.

Meaning-based question

> "How does authentication work?"

Vector search is useful.

Exact-term question

> "Where is `JwtService` used?"

BM25 is useful.

Mixed question

> "How does `JwtService` handle token expiration?"

Both are useful.

```
BM25
→ Finds exact `JwtService` references

Vector Search
→ Finds chunks discussing token expiration/authentication
```

Therefore:

> **DevDocs V1 will use Hybrid Search: BM25 + Vector Search.**

Neither retrieval strategy replaces the other.

---

6. Initial Top-K Configuration

Top-K means:

> "Return the K highest-ranked results."

For DevDocs V1, the initial configuration will be:

```
BM25 Search       → Top-K = 10
Vector Search     → Top-K = 10
```

Therefore, the initial retrieval stage can produce:

```
BM25
  ↓
10 candidates

Vector
  ↓
10 candidates
```

Some results may overlap.

The combined candidate pool can therefore contain up to approximately 20 unique chunks.

These values are initial configuration values and can be tuned later using retrieval evaluation.

---

7. RRF — Reciprocal Rank Fusion

BM25 and vector search produce separate rankings.

For example:

```
BM25:

1. auth.py
2. jwt.py
3. login.py
4. middleware.py
5. user.py
```

```
Vector:
```

```
1. middleware.py
2. auth.py
3. security.py
4. jwt.py
5. login.py
```

We need to create one unified ranking from these two rankings.

This is where **RRF (Reciprocal Rank Fusion)** is used.

7.1 What RRF Does

RRF combines the rankings from multiple retrieval systems.

It primarily uses the **rank/position** of each result rather than directly comparing the raw scores from BM25 and vector search.

This is important because BM25 scores and vector similarity scores are different types of values and cannot simply be added together.

RRF gives higher-ranked results more weight.

If the same chunk appears highly in both retrieval systems, it receives contributions from both rankings.

For example:

```
auth.py
BM25 rank   = 1
Vector rank = 2
```

This makes it a strong candidate in the combined ranking.

7.2 RRF Formula

Conceptually:

```
RRF score = Σ 1 / (k + rank)
```

where `k` is a constant used to control the influence of rank.

The exact mathematical value is an implementation detail.

The important concept is:

```
Higher rank → more contribution
Lower rank  → less contribution
```

7.3 RRF Does Not Replace Reranking

RRF and reranking have different responsibilities.

RRF

Combine and rank the results from BM25 and vector search.

Reranker

Carefully evaluate the candidate chunks against the actual question and select the most relevant ones.

Therefore:

```
BM25 ──────┐
           │
           ├──→ RRF → Unified Ranking → Reranker
Vector ────┘
```

RRF does **not need to remove the candidates simply because they came from different searches**.

Its primary purpose is to produce one unified ranking.

---

8. Candidate Pool

After BM25 and vector search:

```
BM25       → up to 10
Vector     → up to 10
```

RRF combines them into one candidate ranking.

Example:

```
BM25:
A B C D E
```

```
Vector:
C A F B G
```

RRF produces something like:

```
A
C
B
F
D
G
E
```

The exact ranking depends on the ranks returned by the two retrieval systems.

The result is now a **single candidate pool/ranking**.

---

9. Reranking

The next stage is reranking.

Reranking performs a more detailed relevance evaluation of the candidate chunks.

The reranker considers:

```
User Question
      +
Candidate Chunk
```

and produces a relevance score.

For example:

```
Question:
"How does JWT authentication work?"
```

Possible results:

```
auth.py          → 0.96
jwt.py           → 0.94
middleware.py    → 0.89
login.py         → 0.63
payment.py       → 0.08
```

The strongest candidates are then selected.

---

10. Reranking Architecture

The DevDocs V1 default will use a dedicated reranking model.

```
Hybrid Search
      ↓
RRF
      ↓
Candidate Pool
      ↓
Cross-Encoder Reranker
      ↓
Best Relevant Chunks
      ↓
LLM
```

The reranker is intentionally separate from the answer-generating LLM.

The reranker does not generate the final response.

Its responsibility is only:

> **Determine which retrieved chunks are most relevant to the user's question.**

---

11. Optional Jev Reranker

DevDocs will also be designed so that an alternative relevance-scoring model such as **Jev** can be experimented with later.

The architecture should not require changes to the retrieval pipeline when switching rerankers.

Conceptually:

```
Candidate Pool
      ↓
Reranker Interface
      ↓
┌──────────────────────┐
│ Cross-Encoder        │ ← Default
│ Jev                  │ ← Optional
└──────────────────────┘
```

```
      ↓
Final Relevant Chunks
```

The default will remain the dedicated reranker.

Jev will be treated as an experimental alternative that can be enabled when usage credits are available.

---

12. Configurable Reranker

The reranker implementation should be selected through configuration rather than by manually changing the retrieval code.

Example:

```
RERANKER_PROVIDER=cross_encoder
```

To experiment with Jev:

```
RERANKER_PROVIDER=jev
```

The retrieval pipeline remains unchanged.

Conceptually:

```
Hybrid Search
      ↓
RRF
      ↓
Reranker Interface
      ↓
Configuration decides implementation
      ↓
Cross-Encoder / Jev
      ↓
Final Chunks
```

This makes it possible to experiment with different reranking approaches without redesigning the retrieval system.

---

13. Final Top-K

The initial configuration for the reranker will be:

```
Reranker → Top-K = 5
```

This means the reranker can select up to five highly relevant chunks.

The value is a starting point rather than a permanent magic number.

It can be adjusted later based on retrieval evaluation.

Therefore:

```
BM25       → Top-K 10
Vector     → Top-K 10
      ↓
     RRF
      ↓
Candidate Pool
      ↓
Reranker
      ↓
Final Top-K 5
```

---

14. Project Metadata Filtering

Retrieval must respect DevDocs' project isolation architecture.

The mandatory filter is:

```
project_id = current_project_id
```

This prevents chunks from another project from being retrieved.

The retrieval system must also only use files that are ready for RAG.

Therefore the effective retrieval eligibility is:

```
project_id = current_project_id
AND
ingestion_status = READY
```

This prevents files that are still processing or have failed ingestion from entering the RAG context.

---

15. Additional Metadata Filters

For V1, no additional mandatory retrieval filters are required.

Existing metadata includes:

```
project_id
file_id
file_path
file_type
```

Additional filters such as:

```
language
directory
file category
```

can be introduced later if a real product need appears.

The V1 retrieval system should remain simple.

---

16. Handling No Relevant Results

DevDocs should not ask the LLM to answer using unrelated context when retrieval finds nothing sufficiently relevant.

There are two possible situations:

No candidates

```
Search
  ↓
No results
```

Candidates exist but are not relevant

```
Search
  ↓
Candidate chunks
  ↓
Reranker
  ↓
All relevance scores below threshold
```

Both should result in a retrieval failure/fallback rather than an unsupported answer.

Example user message:

> **I couldn't find relevant information in this project to answer that question.**

The system should not invent an answer from unrelated project content.

---

17. Relevance Threshold

The reranking stage should eventually use a relevance threshold.

Conceptually:

```
Reranker
    ↓
Relevance scores
    ↓
Compare against threshold
    ↓
Sufficiently relevant?
   /
 Yes        No
 ↓           ↓
Continue    No relevant context
to LLM      → User fallback
```

The exact threshold depends on the selected reranking model and should be determined experimentally rather than hardcoded arbitrarily.

---

18. Duplicate Handling

Duplicate chunks should not be sent to the LLM.

Duplicates can occur because:

1. The same chunk is returned by BM25 and vector search.
2. Different chunks overlap heavily.

Example:

```
BM25:
auth.py chunk 3
```

```
Vector:
auth.py chunk 3
```

This should represent **one chunk**, not two.

RRF naturally works with the same chunk appearing in multiple rankings as a single candidate with contributions from both rankings.

---

19. Overlapping Chunks

Chunks can also overlap.

For example:

```
Chunk A → lines 20–50
Chunk B → lines 45–75
```

Sending both may waste context because they contain overlapping information.

Therefore, after reranking, DevDocs should perform basic context cleanup/deduplication before sending the final context to the LLM.

V1 should keep this simple rather than implementing complex semantic deduplication.

---

20. Context Token Budget

The number of final chunks alone should not determine how much information is sent to the LLM.

For example:

```
Chunk 1 → 1,000 tokens
Chunk 2 → 1,500 tokens
Chunk 3 → 800 tokens
Chunk 4 → 1,200 tokens
Chunk 5 → 1,000 tokens
```

Five chunks would already contain approximately:

```
5,500 tokens
```

The LLM prompt also needs space for:

- System instructions
- Conversation context
- User question
- Retrieved context
- Expected response

Therefore DevDocs should use a **retrieved-context token budget**.

The final Top-K should be treated as a **maximum**, not a guarantee.

Conceptually:

```
Reranker
    ↓
Top 5 candidates
    ↓
Context token budget
    ↓
Add highest-ranked chunks
until budget is reached
    ↓
Final context
    ↓
LLM
```

For example:

```
Maximum retrieved context ≈ 4,000–6,000 tokens
```

The exact budget will depend on the selected LLM and will be tuned during implementation.

---

21. Retrieval Does Not Equal LLM Token Usage

BM25 and vector retrieval can examine many candidates without sending all of them to the LLM.

For example:

```
BM25 → 10
Vector → 10
       ↓
      RRF
       ↓
Candidate pool
       ↓
```

```
Reranker
       ↓
5 or fewer relevant chunks
       ↓
LLM
```

Only the final context contributes directly to the LLM's input-token usage.

Therefore, retrieving a larger candidate pool does not automatically mean that the LLM receives a larger prompt.

The purpose of the retrieval pipeline is to progressively reduce the information before it reaches the LLM.

---

22. Retrieval Evaluation

DevDocs needs a way to determine whether retrieval is actually working well.

A small evaluation dataset should be created containing representative project questions and the expected relevant files/chunks.

Example:

```
Question:
How does authentication work?

Expected relevant files:
auth.py
jwt.py
```

Another:

```
Question:
How are orders created?

Expected relevant files:
order_service.py
order_controller.py
```

Another:

```
Question:
How is the database connection created?

Expected relevant files:
```

```
database.py
config.py
```

These questions can be repeatedly tested against the retrieval pipeline.

---

23. Retrieval Evaluation Metrics

Later, DevDocs can measure retrieval quality using metrics such as:

- Recall@K
- Precision@K
- MRR
- Reranker effectiveness

The purpose is to answer questions such as:

> Did the retrieval system actually retrieve the information needed to answer the question?

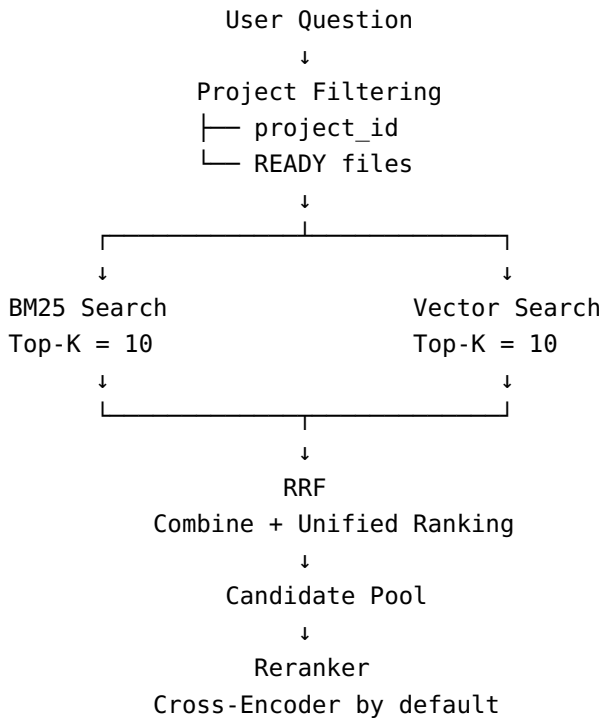
These metrics do not need to be part of the user-facing application.

They are primarily development and evaluation tools.

---

24. Complete Search & Retrieval Pipeline

The finalized V1 architecture is:



```
          /
         /
Cross-Encoder       Jev
  default        experimental
              ↓
      Relevance Threshold
              ↓
          Deduplication
              ↓
      Context Token Budget
              ↓
      Final Top-K ≤ 5
              ↓
          Relevant Context
              ↓
             LLM
```

---

25. V1 Configuration Summary

Component	V1 Decision
Retrieval strategy	Hybrid
BM25	Enabled
Vector search	Enabled
BM25 Top-K	10
Vector Top-K	10
Combination method	RRF
Reranker	Cross-encoder
Alternative reranker	Jev
Reranker selection	Configuration-based
Final Top-K	Maximum 5
Project filter	`project_id`
File eligibility	`READY` only
Additional metadata filters	Not required for V1
No relevant results	User-facing fallback
Relevance threshold	Required, exact value to be tuned
Duplicate handling	Required
Overlap handling	Basic cleanup

Component	V1 Decision
Context limit	Token budget
Retrieval evaluation	Required during development

---

26. Core Principles

1. Retrieve broadly, then narrow

```
BM25 + Vector
      ↓
Candidate pool
      ↓
RRF
      ↓
Reranker
      ↓
Final context
```

2. Use both exact matching and semantic matching

BM25 finds important words.

Vector search finds similar meaning.

3. RRF combines; it does not replace reranking

RRF creates a unified ranking.

Reranking performs deeper relevance selection.

4. Keep irrelevant context away from the LLM

The LLM should receive only the most relevant project context that fits within the context budget.

5. Project isolation is mandatory

Retrieval must never cross project boundaries.

```
current_project_id
        ↓
only that project's READY chunks
```

6. Failed or processing files are not retrieval sources

Only successfully ingested (`READY`) files participate in RAG retrieval.

7. Do not hallucinate when retrieval fails

If no sufficiently relevant context is found, DevDocs should tell the user rather than inventing an answer.

8. Retrieval quality must be measurable

A retrieval evaluation dataset should be used to tune Top-K, reranking, thresholds, and other retrieval parameters.

---

27. Final Mental Model

The easiest way to remember DevDocs Search & Retrieval is:

```
BM25
"Find chunks containing the important terms."

Vector Search
"Find chunks with similar meaning."

RRF
"Combine both rankings."

Reranker
"Which of these candidates are actually relevant?"

Deduplication
"Don't send the same information twice."

Context Budget
"Don't send too much information."

LLM
"Now answer the user's question using the selected context."
```

Therefore:

> **Search & Retrieval is responsible for finding and selecting the right project knowledge. It does not generate the final answer.**